

Building Software the Right Way

A Verified Engineering Standards Reference

Compiled from a real, working production codebase (Harbor), verified against its own history and its own Architecture Decision Records, and extended with current, authoritative industry standards.

For use as a starting standard on any new software project.

How to Use This Document

This document is built differently from a typical engineering handbook. Every practice in Parts 1 through 9 was not written from general knowledge about what a good project should do. It was extracted and verified directly against a real, working, live production codebase, its actual commit history, its own persistent build log, and its own formally recorded architecture decisions. Where that project was inconsistent, that inconsistency is stated plainly rather than smoothed into a clean rule that does not actually hold.

Part 10 extends this with current, authoritative, external industry standards (OWASP's own security risk rankings, the twelve-factor application methodology, and modern testing and versioning conventions) so this document reflects not just one project's own good habits, but the broader, current consensus of the wider software industry.

Part 11 is a condensed, practical checklist meant to be pinned somewhere visible on a new project, not read once and filed away.

Each section ends with a portable rule, a version of the practice written generally enough to apply to any project, in any language, on any stack.

Table of Contents

Part 1: Code Style and Conventions

What this looks like in practice

A new feature follows one repeated pipeline, evidenced across dozens of real features: schema change first, applied via a migration and independently re-verified with a direct query, never trusted from the migration's own exit code. Business logic lives in a single new file named for the concern, one file, one real responsibility. Thin API route handlers call straight into that logic and translate its typed errors into HTTP statuses. UI reuses shared components rather than reinventing them. A matching pair of test scripts closes out the feature.

A real, consistent naming taxonomy

- `resolve<Thing>` — a function that looks up who is asking and returns their scoped resource
- `compute<Thing>` — a pure business-rule calculation with no side effects
- `validate<Thing>Input` — pure input validation, called before anything is trusted
- `clamp<Thing>` — defensively bounding a value (especially an AI-returned one) before it reaches storage

Error classes are always a named, specific class ending in `Error`, never a generic thrown string, so a route handler can map each one to a precise, correct HTTP status.

File organization as a real, enforced discipline

Role-based folders are kept as plain folders, not framework route groups that hide the real URL. This is a documented, corrected mistake worth naming directly: an early draft used a syntax that made two different roles silently resolve to the same path, which crashed the app. The fix became a standing rule against ever repeating that pattern.

Linting and formatting, mechanically enforced, not just encouraged

A typecheck pass and a lint pass are a hard gate before any feature is called done, every time, not occasionally. One specific rule (state updates during a render effect) was caught and fixed repeatedly across unrelated features, always the same two correct ways, never suppressed with an inline disable comment.

One mechanically enforced style rule worth naming specifically because it is unusual to see actually verified rather than just requested: a project-wide ban on a specific punctuation mark, checked by a literal text search, not just stated as a style preference.

An honest inconsistency worth naming

Whether a new categorical field becomes a strict, closed enumeration or a plain, open text field is a deliberate axis, not one universal rule: a field expected to grow new values later stays open text specifically so a new value never needs a schema migration, while a field whose vocabulary is genuinely closed becomes a real enum. Both are correct within their own reasoning, but this means two different-looking patterns exist for what looks, at a glance, like the same kind of decision. A newcomer should be told this is deliberate, not left to guess.

Portable rule: Give every feature the same shape (schema, pure logic, thin routes, UI, two tiers of tests) so anyone can find anything by knowing the pattern. Name functions by what they do to a value, not by their implementation detail. Enforce formatting and type-safety mechanically, as a real gate, not through review discipline alone.

Part 2: The “Null Over Guessed” AI Discipline

What this looks like in practice

Everywhere an AI model's output feeds a real decision or a stored value, the surrounding code either verifies that output against a checkable fact, or accepts an honest null or empty result rather than coercing a plausible-looking guess into place.

Real, concrete evidence of this pattern

- Data extraction is explicitly instructed to return null rather than guess a fact it cannot actually determine, and carries its own confidence field; low confidence blocks the workflow entirely rather than proceeding on a guess.
- A categorical value (like a language code) is never accepted as free text anywhere a real reference table already exists, so a model can never invent a value that is not real.
- A count obtained by parsing a file exactly is kept clearly distinct from a count only estimated by a model, and the estimate is labeled as an estimate everywhere it is shown, never presented with false precision.
- Every AI-returned number that reaches a numeric database column passes through a defensive bounding function first, with the reasoning stated directly in code: a schema guides a model, it does not guarantee it.

- A model is never asked to compute an absolute date from its own implicit sense of “today.” It is asked for a relative duration, which the server then adds to the real current time itself.
- A drafting pipeline never trusts a model's own claim that it preserved every required token or placeholder. A separate, independent check re-verifies this against the source and forces the confidence score down, naming exactly what was found missing, if the check fails.
- A classification feature never trusts a name or company string written by the model itself. The schema it is allowed to return does not even define those fields, only an identifier; the real display text is always re-fetched from the original, already-verified data. An identifier the model invents that was never in the real input is silently dropped, never shown with a caveat attached.
- A transcription pipeline discovered, live, that genuinely silent input caused a model to hallucinate a plausible-sounding result instead of returning an honest empty one. The fix was not a better prompt. It was an independent, deterministic, code-level check that runs before the model is ever asked, on both the sending and receiving side.
- An AI-driven action system is only ever allowed to choose from a small, closed, human-authored list of real actions by identifier. It can never invent a new action, a new destination, or a new target, and the chosen identifier is independently re-validated against the real list before it is ever acted on.

Portable rule: Wherever an AI-generated value feeds a real decision or a stored field, verify or bound it in code against something checkable, and let it come back honestly empty rather than forcing a plausible-looking answer, especially anywhere a ground truth already exists that can catch or override a wrong guess.

Part 3: Testing Philosophy and Structure

Two deliberate tiers, not one

Pure tests touch no database and no network. They exist specifically to pin down boundary conditions and defensive logic in functions that were deliberately pulled out to be pure and independently testable, even when a function is only ever called from one place in the whole codebase.

Live integration tests run against real infrastructure, a real database, real storage, real external calls, never mocks. This is a stated, deliberate rule: no mocking framework is

used at all. Fixtures are disposable, created and cleaned up automatically, and that cleanup is proven idempotent by running the same script twice in a row.

For anything with a real, user-facing claim, a third tier exists: live browser-driven tests against a running instance, with real screenshots saved as durable, checkable evidence, not just a pass or fail status.

The bar for “done”

A clean type check. A clean lint pass. A clean build. The feature's own pure and integration suites passing. And whenever a widely shared file is directly edited, a full regression sweep re-running every other suite that depends on it, not just the new feature's own tests.

Adversarial fixture testing, a named, repeatable technique

Two concrete forms of this are used, both deliberately trying to break the system rather than merely confirming it works.

- The near-miss trap: when testing whether a classification correctly ignores irrelevant data, the test batch deliberately includes a candidate designed to tempt a wrong answer, run several independent times with a majority-vote bar, since a live model is honestly not perfectly deterministic and the test says so rather than pretending otherwise.
- The attack-payload proof: rather than assert that a security setting is merely configured correctly, a real, live, malicious payload is handed directly to the system with the defense in place, proving the defense actually neutralizes it, not just that a flag was set somewhere.

The most repeated idea in this whole discipline

"This looks correct, versus, I ran it against real data and watched it work."

A hard, named line is kept between these two claims, and the stronger one is never claimed without the weaker one's evidence actually in hand.

An honest inconsistency worth naming

Not every feature gets full three-tier coverage; browser-driven tests are reserved for genuine user-facing claims. At least one such test was honestly reported as unable to complete in a given session due to a real, local resource constraint, rather than silently marked as passing.

Portable rule: Write pure tests for extracted business logic, live tests against real infrastructure for anything touching a database or an external service, and never claim a stronger form of proof than you actually gathered. When testing that a system correctly rejects something, include a case specifically designed to fool it, not just an obviously wrong one.

Part 4: Security Practices

Isolation as the single most explicitly named rule

Every business record carries an owner or tenant identifier with no exception, and every query against it must filter by that identifier. The one deliberate exception in a real system reviewed here (a genuinely global identity, scoped only through a join table) is documented and treated as the load-bearing exception it is, never as an oversight.

A request for a resource that belongs to someone else returns the identical response shape as a request for a resource that simply does not exist, so a probe can never learn which case it actually hit.

A real, live isolation incident, and the lesson it produced

Two disposable test accounts were discovered, via an external notification, to both be connected to the wrong external account entirely, a human setup mistake, not a code defect. The correct response was: disconnect both through the real, existing mechanism (never a raw data deletion), find and fix a second, related misconfiguration the investigation surfaced along the way, and, most importantly, write a permanent regression test whose ability to actually catch the problem was directly proven, not assumed, by deliberately recreating the failure and confirming the new test actually failed against it before the fix, then passed after.

"When you fix an isolation incident, write a test that is proven to catch the specific failure, not one that merely passes."

Secret handling

Real secrets live only in a local, gitignored environment file and a separate, dedicated secrets store for the deployed environment; an example file documents every required variable with no real values ever committed. A hard-learned rule: never pass a secret through a shell command whose output could echo it, since an unquoted special character in a credential can be misinterpreted by a shell and printed in full, in plain

view. The fix that became standing practice: write a secret to a disposable file and reference the file, never let it pass through a command line or a reply directly.

Privilege and access design

A platform-owner allowlist deliberately reads from the deployment environment itself, never from an in-product, user-editable field, specifically because a mutable field is writable through normal application code paths and a true owner-level grant must only be settable from somewhere only the deployment holds.

Rate limiting is enforced at the data layer with a real, atomic conditional update, not in memory, because a modern, serverless, horizontally-scaled environment shares no memory across invocations.

An honest inconsistency worth naming

Not every credential exposure received the same urgency of response. Some were rotated immediately; at least one was named plainly as a real, open item left for a human to act on, rather than force-rotated automatically. The discipline of disclosing a problem is universal here; the discipline of immediately fixing every one of them is judgment-dependent, and that is stated honestly rather than implied to be uniform.

Portable rule: Treat isolation between owners or tenants as something the database itself enforces, not something the application merely respects by convention. Make every access-denied response indistinguishable from a not-found one. Never let a secret pass through a shell command that could echo it. And when an isolation incident happens, fix the root cause and write a test proven, by deliberately trying to fail it, to actually catch the same failure again.

Part 5: Error Handling and Failure Modes

Distinguishing a real bug from a transient environment issue

Real infrastructure and networks fail in genuinely different ways, and each failure is investigated to a real, specific, understood root cause before being dismissed as flakiness or blamed on the application code. Lumping every unexplained failure together as “just flaky, retry it” is treated as a real discipline failure, not a shortcut.

A genuinely progressing process was once mistakenly killed based on a misleading signal (very low resource usage, which is actually normal for a process waiting on a network call, not evidence that it is stuck). This was reported plainly as an investigative mistake the moment it was understood, not quietly re-run and left unmentioned.

Errors are specific, never generic

A dedicated error-description helper was built specifically because a generic wrapper made two genuinely different underlying failures look identical, and that ambiguity made a real bug undiagnosable until the wrapper itself was fixed.

Retries are scoped narrowly, never applied blindly

A retry mechanism checks the actual, specific error class and fails fast on anything a retry could never fix (a data error, a constraint violation), while only retrying genuinely transient failures. It is never wrapped around a multi-step transaction or anything with a real double-write risk, because blindly retrying those carries a real risk of duplicating an action or a charge.

A secondary system must never break a primary action

Every best-effort, secondary capability (a notification, a sync to an external system, an analytics event) is wired to log and continue on failure, never to block the real, primary action it rides alongside. The strongest version of this guarantee is directly tested, not just assumed: a simulated failure is injected into the secondary system, and the test confirms it never propagates outward to break the primary action.

An honest inconsistency worth naming

Not every write in the system is wrapped in a retry safety net. At least one gap was explicitly, deliberately left unwrapped and named as an accepted, low-severity residual risk, because wrapping it would introduce a different, worse risk (retrying a non-idempotent write). The real principle held is not “wrap everything,” it is “wrap only what is genuinely safe to retry, and say plainly what was not.”

Portable rule: Log failures with enough real context to actually diagnose them, never a generic message. Give every meaningful failure its own specific, named type. Never let a best-effort, secondary action's failure break the primary action it rides alongside. And before blaming your own code for a flaky-looking failure, investigate the environment to a real, specific, named root cause first.

Part 6: CI/CD and Deployment Practices

One real, automated deployment path

Deployment flows directly from the version-control system: every change to the main line auto-deploys to production, and every other branch gets its own disposable preview

environment with its own isolated data. This is a real, current, industry-standard pattern, not unique to any one project.

Holding the bar yourself when a formal gate is off

If continuous integration is disabled for a real, deliberate, disclosed reason (most commonly cost, at an early stage with no formal review process yet to serve), the exact same quality bar it would have enforced (type safety, linting, a clean build) must still be held manually, every time, not relaxed simply because nothing is automatically checking it.

Environment variables need a real redeploy

A specific, hard-learned bug pattern: changing a configuration value in a deployment dashboard and assuming it takes effect immediately, when an already-running deployment needs a genuine redeploy to actually pick up the new value. The correct discipline is to test with both the old and new value, before and after a real redeploy, and confirm the behavior actually changed, rather than trusting the dashboard's own confirmation alone.

Verify live, not just merged

After any production-affecting change, the deployed system itself is independently checked, a real request to the real deployed address, a real query against the real production data, or a real status check against the deployment platform's own reporting, rather than trusting a successful merge alone as proof anything actually changed for real users.

Avoid standing up permanent infrastructure for a temporary problem

Before adopting a heavier, more permanent piece of infrastructure (a persistent background worker, a message queue), the real, underlying problem is named honestly first. If the actual problem is a scheduling or cost constraint rather than a genuine queueing need, a much simpler scheduled trigger is the right fix, and standing up permanent infrastructure to solve a smaller problem is treated as a real, avoidable overreach.

Portable rule: Deploy through one real, automated path, and hold the exact same quality bar yourself even when a formal gate is turned off by deliberate choice. Verify every meaningful change is actually live on the real, deployed system through an independent check, not just a successful merge. When you disable part of your own pipeline, say so plainly and explain the tradeoff, rather than letting it quietly stop mattering.

Part 7: Documentation Practices

One persistent, append-only project memory

A single, chronologically ordered file is kept as the project's own memory, one that a completely fresh contributor, human or otherwise, is expected to read cold and be fully oriented by. It functions as a real, running build log, not a status summary: every real decision, every real bug found, and every honest limitation is appended as its own entry and never rewritten to quietly erase history. When an earlier entry turns out to have been wrong, the correction is written in plainly, next to the original claim, not silently edited away.

Formally recorded decisions are treated as binding

A specific, numbered decision record is cited as the actual authority behind a real design choice far more often than a vague appeal to general best practice. When code or a comment is later found to have drifted from what a decision record actually says, the decision record wins, and the code or comment is corrected to match it, never the other way around.

Provenance is stated honestly

Anything transcribed or summarized from an original source that was never fully available carries an explicit note saying so, rather than being presented as if it were the complete original.

An honest inconsistency worth naming

The discipline here is strong but not flawless. More than once, a claim written into the persistent memory turned out, on direct re-check, to be false. Each was caught and corrected explicitly rather than left standing, but the honest fact is that the file's own historical claims are not perfectly reliable on first write. Their reliability comes from the fact that they get checked and corrected later, not from never being wrong in the first place.

Portable rule: Keep one persistent, append-only project memory that a cold start can read and be fully oriented by, including open items and past mistakes, not just a status summary. Treat formally recorded decisions as binding, and fix code or comments that drift from them rather than the other way around. Never quietly overwrite a documented mistake; correct it visibly, right next to where it was made.

Part 8: Scope Discipline

A real, named commitment to sequencing

A dedicated decision record explicitly commits a project to finishing one thing well before starting the next, and it is cited by name, repeatedly, as the actual reason something is not being built yet, not just a vague, unexplained sense of caution.

A separate, long-form vision document is explicitly labeled as context only, never a task list, with an explicit instruction not to build toward it ahead of what the current, real, immediate need actually calls for.

Plan first, build second, always in that order

Before any nontrivial code is written, a real, written plan is produced and explicitly confirmed before implementation starts. Alternatives that were considered and rejected are written down with their real reasons, not silently omitted, so a future reader understands not just what was built, but what was deliberately not built, and why.

Deliberately parked work gets a concrete trigger, never a vague later

Work that is correctly not being built yet is gated on a specific, checkable condition, a certain number of independent users, a specific piece of infrastructure existing first, not an open-ended “someday.”

An honest inconsistency worth naming

This discipline scales with the size of the decision. It is applied very consistently to big, structural choices, and less formally to small, in-session decisions that are sometimes made and built in the same sitting once confirmed. The rigor is not applied uniformly to every line of code, and that is stated plainly rather than overclaimed.

Portable rule: Write and get explicit agreement on a real plan before building anything nontrivial. State plainly what you are deliberately choosing not to build and why, with a concrete condition for revisiting it, never a vague later. Treat a long-term vision as something to consult, never a backlog to start pulling from ahead of the current, real need.

Part 9: The Honest-Reporting Pattern

The single most valuable, most unusual practice here

A mistake, a wrong theory, a stale claim, or a misdiagnosis is reported exactly as plainly as a success would be, the moment it is found, rather than quietly fixed and left unmentioned.

Why this matters as a deliberate practice, not just good manners

A persistent project record is only useful as ground truth for the future if every claim in it can be trusted at face value. The moment even one entry is a face-saving overclaim, the whole record's authority collapses, because a future reader can no longer tell which parts to trust without independently re-verifying everything, which defeats the entire point of keeping the record in the first place.

This exact discipline is also what catches a real, silently-shipping bug before it reaches production: an investigation willing to conclude a comment or a claim was simply wrong, instead of assuming it must be right and quietly building around it, is what surfaces the real problem in time.

Portable rule: State a mistake, a wrong theory, or a stale claim exactly as plainly as you would state a success, the moment you find it. Distinguish clearly between what you believe is true and what you have actually proven. Correct a wrong claim in a shared record openly, right next to where it was made, never quietly. The value of a project's history depends entirely on every entry in it being trustworthy.

Part 10: Supplementary Industry Standards

The nine practices above were verified against one real project's own history. This section adds current, external, industry-wide standards, so this document reflects the broader consensus of the software industry, not only one project's good habits.

10.1 The OWASP Top 10 (2025 edition, the current standard as of 2026)

The Open Web Application Security Project's Top 10 is the industry's most widely cited baseline for web application security risk, built from data across hundreds of thousands of real applications. Any new project should be checked against all ten categories as a genuine minimum bar, not treated as optional reading.

- A01: Broken Access Control — the current, single most common real-world flaw, appearing in the vast majority of applications tested. Now also absorbs Server-Side Request Forgery as a sub-category.
- A02: Security Misconfiguration — insecure defaults, unnecessary enabled features, and exposed error detail; found in some form in effectively all applications tested.
- A03: Software Supply Chain Failures — a 2025 addition, expanding the older “vulnerable components” idea to cover the full chain: compromised dependencies, compromised build systems, and compromised distribution.
- A04: Cryptographic Failures — weak, missing, or misapplied encryption for data in transit and at rest.
- A05: Injection — now understood broadly, well beyond classic SQL injection, to include NoSQL, ORM-layer, and expression-language injection.
- A06: Insecure Design — flaws baked into the architecture itself, not fixable by a patch, requiring real threat modeling before code is written.
- A07: Authentication Failures — weak recovery flows, missing multi-factor support, and session-handling weaknesses; by 2026, password-only authentication is no longer considered acceptable for anything sensitive.
- A08: Software and Data Integrity Failures — trusting unsigned or unverified updates, plugins, or serialized data.
- A09: Security Logging and Monitoring Failures — insufficient logging to detect and respond to a real breach in progress.

- A10: Mishandling of Exceptional Conditions — a 2025 addition, covering improper error handling and systems that fail open (permissive) instead of failing closed (restrictive) when something goes wrong.

Note for anything built on a large language model specifically: the separate OWASP LLM Top 10 exists for exactly this case, centered on prompt injection and the real blast radius of an autonomous agent holding genuine credentials or the ability to take real actions.

Portable rule: Check every new project against all ten current OWASP categories as a genuine minimum bar, and if the project uses an LLM with real tool access, check it separately against the OWASP LLM Top 10 too.

10.2 The Twelve-Factor App Methodology

A widely adopted set of principles for building software-as-a-service applications that are portable, scalable, and maintainable across environments. Highly relevant to anything deployed the way modern web applications typically are.

- One codebase, tracked in version control, with many deploys from it.
- Explicitly declare and isolate dependencies, never rely on anything assumed to exist on the host system.
- Store configuration in the environment, never hardcoded in the codebase.
- Treat backing services (a database, a queue, a cache) as attached resources, swappable without a code change.
- Strictly separate the build, release, and run stages.
- Execute the app as one or more stateless processes, sharing nothing that isn't in a backing store.
- Export services via port binding, rather than relying on runtime injection of a webserver.
- Scale out via the process model, not by making a single process larger.
- Maximize robustness with fast startup and graceful shutdown.
- Keep development, staging, and production as similar as possible.
- Treat logs as event streams, never managed as files by the application itself.
- Run admin and maintenance tasks as one-off processes, in the same environment as the app.

Portable rule: Configuration lives in the environment, never in code. Treat every backing service as swappable. Keep development and production as alike as realistically possible.

10.3 The Testing Pyramid

A widely used model for how a healthy test suite should be shaped: many fast, cheap, isolated unit tests at the base, a smaller number of integration tests in the middle verifying real components work together, and a small number of full end-to-end tests at the top verifying genuine user-facing flows. The shape matters because each layer up is slower, more brittle, and more expensive to maintain; a suite shaped like an inverted pyramid (mostly slow end-to-end tests, few real unit tests) is a well-known, common failure pattern that makes a codebase slow and painful to change safely.

Portable rule: Keep most of your tests fast, isolated, and cheap. Reserve slow, full end-to-end tests for the handful of flows where only a real, full-system proof will do.

10.4 Conventional Commits and Semantic Versioning

Two widely adopted, complementary conventions worth adopting from day one on any new project, since retrofitting them onto an existing history is far harder than starting with them.

Conventional Commits: a lightweight, structured format for commit messages (feat:, fix:, chore:, docs:, and similar prefixes) that makes a project's history genuinely readable and machine-parseable, enabling automatic changelog generation and automatic version bumping directly from commit history.

Semantic Versioning: a widely adopted MAJOR.MINOR.PATCH numbering scheme where a breaking change increments MAJOR, a backward-compatible new feature increments MINOR, and a backward-compatible fix increments PATCH, giving anyone depending on a project a reliable, predictable signal about the real risk of upgrading.

Portable rule: Adopt a structured commit message convention and semantic versioning from the very first commit, not after the history has already grown too large to retrofit cleanly.

Part 11: Quick Reference Checklist

A condensed, practical checklist. Meant to be kept visible during real work, not read once and archived.

Before writing any nontrivial code

- Write a real, explicit scope. Get it confirmed before implementation starts.
- Name the alternatives you considered and rejected, and why.
- Check whether a formally recorded decision already governs this area. If so, it wins over a fresh guess.

While building

- One file, one real responsibility, for new business logic.
- Name functions by what they do to a value (resolve, compute, validate, clamp), not by implementation detail.
- Give every meaningful failure its own specific, named error type.
- Never let a secondary action's failure break the primary action it rides alongside.
- Never let an AI-generated value reach storage or a decision unverified. Prefer an honest null over a forced guess.
- Never let a secret pass through a shell command, a log line, or a reply where it could be echoed.

Before calling anything done

- A clean type check.
- A clean lint pass.
- A clean build.
- Pure tests for extracted business logic, passing.
- Live integration tests against real infrastructure, passing, no mocks.
- A full regression sweep if you touched a widely shared file.
- For a genuine security-relevant claim, an adversarial test that actually tries to break it, not just a configuration check.
- For a genuine user-facing claim, real evidence (a screenshot, a live browser test), not just “the code looks right.”

Before and after deploying

- Hold the same quality bar yourself even if an automated gate is off.
- After any config change in a deployment dashboard, confirm with a real redeploy, don't assume it's live.
- Independently verify the change is live on the real, deployed system. A successful merge is not proof by itself.

When something goes wrong

- Investigate an unexplained failure to a real, specific, named root cause before blaming code or dismissing it as flakiness.
- If it's a real isolation or security incident, fix the root cause and write a test proven, by deliberately trying to fail it, to actually catch the same failure again.
- Report the mistake exactly as plainly as you would report a success. Correct a wrong claim in the shared record openly, next to where it was made.

Security, minimum bar

- Every business record is scoped to its owner with no exception, enforced at the query level, not by convention.
- An access-denied response looks identical to a not-found response.
- Configuration lives in the environment, never hardcoded.
- Checked against all ten current OWASP Top 10 categories.
- If the project uses an LLM with real tool access, separately checked against the OWASP LLM Top 10.

End of document.